Practical No 01

A) Create an MVC application to perform basic Arithmetic Operations.

Project Name: ArthimeticMVC

Project Type: MVC Application

Program Code:

I. Filename: Index.cshtml

```
@model ArithmeticMVC.Models.ArithmeticModel
@{
    ViewBag.Title = "Arithmetic Operations";
}

<h2>Arithmetic Operation</h2>
@using (Html.BeginForm())
{
    <table>
        <tr>
            <td>Frist Number</td>
            <td>@Html.TextBoxFor(m=>m.Number1)</td>
        </tr>

        <tr>
            <td>Second Number</td>
            <td>@Html.TextBoxFor(m=>m.Number2)</td>
        </tr>

        <tr>
            <td>Operation</td>
            <td>
                @Html.DropDownListFor(m=>m.Operation,
                new SelectList(new[]
                {
                    "Add",
                    "Subtract",
                    "Multiply",
                    "Divide"
                }))
            </td>
        </tr>

        <tr>
            <td></td>
            <td>
                <input type="submit" value="Calculate"/>
            </td>
        </tr>
    </table>
}

@if(!string.IsNullOrEmpty(Model.ErrorMessage))
{
    <h3 style="color:red">@Model.ErrorMessage</h3>
}
else if(Model.Operation!=null)
{
    <h3>Result=@Model.Result</h3>
}
```

II. Filename: HomeController.cs

```
public class HomeController : Controller
{
    public ActionResult Index(ArithmeticModel model)
    {
```

```
        if (ModelState.IsValid)
        {
            switch (model.Operation)
            {
                case "Add":
                    model.Result = model.Number1 + model.Number2;
                    break;
                case "subtract":
                    model.Result = model.Number1 - model.Number2;
                    break;
                case "Multiply":
                    model.Result = model.Number1 * model.Number2;
                    break;
                case "Divide":
                    if (model.Number2 == 0)
                    {
                        model.ErrorMessage = "Divison by zero is not allowed.";
                    }
                    else
                    {
                        model.Result = model.Number1 / model.Number2;
                    }
                    break;
            }
        }
        return View(model);
    }
}
```

III. Filename: ArithmeticModel.cs

```
using System.ComponentModel.DataAnnotations;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;

namespace ArithmeticMVC.Models
{
    public class ArithmeticModel
    {
        public double Number1 { get; set; }
        public double Number2 { get; set; }
        public string Operation { get; set; }
        public double Result { get; set; }
        public string ErrorMessage { get; set; }
    }
}
```

B) Create an application to print Floyd's triangle till n rows in C#.

Project Name: Floydtriangle

Project Type: Console App

Program Code:

I. Filename: Program.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace FloydTriangle
{
    class Program
    {
        static void Main(string[] args)
        {
            int n, num = 1;

            Console.Write("Enter the number of rows:");
            n = Convert.ToInt32(Console.ReadLine());

            Console.WriteLine("\nFloyd's Triangle:\n");
            for (int i = 1; i <= n; i++)
            {
                for (int j = 1; j <= i; j++)
                {
                    Console.Write(num + " ");
                    num++;
                }
                Console.WriteLine();
            }
            Console.ReadKey();
        }
    }
}
```

C) Write C# code to arrange the name of cities in sorted order. Accept name of 10 cities from the user.

Project Name: CitySorting

Project Type: Console App

I. Filename: Program.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace FloydTriangle
{
    class Program
    {
        static void Main(string[] args)
        {
            int n, num = 1;

            Console.Write("Enter the number of rows:");
            n = Convert.ToInt32(Console.ReadLine());

            Console.WriteLine("\nFloyd's Triangle:\n");
            for (int i = 1; i <= n; i++)
            {
                for (int j = 1; j <= i; j++)
                {
                    Console.Write(num + " ");
                    num++;
                }
                Console.WriteLine();
            }
            Console.ReadKey();
        }
    }
}
```

Practical No 02

A) Write the program with Function Overloading.

Project Name: FunctionOverloading

Project Type: Console app

Program Code:

I. Filename: Program.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace FunctionOverloading
{
    class Calculator
    {
        public int Add(int a, int b)
        {
            Console.WriteLine($"Add(int, int)called");
            return a + b;
        }

        public int Add(int a, int b, int c)
        {
            Console.WriteLine($"Add(int,int,int)called");
            return a + b + c;
        }

        public double Add(double a, double b)
        {
            Console.WriteLine($"Add(double,double)called");
            return a + b;
        }

        public string Add(string a, string b)
        {
            Console.WriteLine($"Add(string,string)called");
            return a + b;
        }
    }

    class Program
    {
        static void Main(string[] args)
        {
            Calculator calc = new Calculator();
            Console.WriteLine("=== Function Overloading Demo ===");
            Console.WriteLine(calc.Add(10, 20)); //30
            Console.WriteLine(calc.Add(10, 20, 30)); //60
            Console.WriteLine(calc.Add(1.5, 2.5)); //4.0

            Console.WriteLine(calc.Add("Hello, ", "World!"));
            Console.ReadKey();
        }
    }
}
```

B) Write the program with Inheritance (all types)

Project Name: Floydtriangle

Project Type: Console App

Program Code:

I.Filename: Program.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace InheritanceDemo
{
    class Animal
    {
        public void Eat() => Console.WriteLine("Animal is eating");
    }

    class Dog : Animal
    {
        public void Bark() => Console.WriteLine("Dog is barking");
    }

    class GrandParent
    {
        public void Method1() => Console.WriteLine("GrandParent method");
    }

    class Parent : GrandParent
    {
        public void Method2() => Console.WriteLine("Parent method");
    }

    class Child : Parent
    {
        public void Method3() => Console.WriteLine("Child method");
    }

    class Shape
    {
        public void Draw() => Console.WriteLine("Drawing a shape");
    }

    class Circle : Shape
    {
        public void Info() => Console.WriteLine("This is a circle");
    }

        class Rectangle : Shape
    {
        public void Info() => Console.WriteLine("This is a rectangle");
    }

    interface IFlyable
    {
        void Fly();
    }

    interface ISwimable
```

```
    {
        void Swim();
    }

    class Duck : IFlyable, ISwimable
    {
        public void Fly() => Console.WriteLine("Duck can fly");
        public void Swim() => Console.WriteLine("Duck can swim");
    }

    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("---Single Inheritance---");
            Dog d = new Dog();
            d.Eat(); d.Bark();

            Console.WriteLine("---Multilevel Inheritance---");
            Child c = new Child();
            c.Method1(); c.Method2(); c.Method3();

            Console.WriteLine("---Hierarchical Inheritance---");
            Circle cir = new Circle(); cir.Draw(); cir.Info();
            Rectangle rec = new Rectangle(); rec.Draw(); rec.Info();

            Console.WriteLine("---Multiple Inheritance(interface)---");
            Duck duck = new Duck();
            duck.Fly(); duck.Swim();
            Console.ReadKey();
        }
    }
}
```

C) Write the program with Constructor Overloading

Project Name: Constructor

Project Type: Console App

Program Code:

I.Filename: Program.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Constructor
{
    class Student
    {
        public Student()
        {
            Console.WriteLine("Default Constructor Called");
        }

        public Student(string name)
        {
            Console.WriteLine("Student Name:" + name);
        }

        public Student(string name, int age)
        {
            Console.WriteLine("Student Name:" + name);
            Console.WriteLine("Student Age:" + age);
        }
    }

    class Program
    {
        static void Main()
        {
            Student s1 = new Student();
            Student s2 = new Student("Shub");
            Student s3 = new Student("Raj",19);
            Console.ReadLine();
        }
    }
}
```

D) Write the program with Interfaces

Project Name: Interface

Project Type: Console App

Program Code:

I.Filename: Program.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Interface
{
    interface IShape
    {
        void Area();
    }

    class Square:IShape
    {
        int side = 5;
        public void Area()
        {
            Console.WriteLine("Area of Square =" + (side * side));
        }
    }

    class Program
    {
        static void Main()
        {
            Square s = new Square();
            s.Area();
            Console.ReadLine();
        }
    }
}
```

E) Write the program with Using Delegates and events

Project Name: Delegatesvents

Project Type: Console App

Program Code:

I.Filename: Program.cs

```
using System;
using System.Text;
using System.Threading.Tasks;

namespace DelegatesEvents
{

    public delegate void NotifyHandler(string message);

    class EventPublisher
    {
        public event NotifyHandler OnDataProcessed;

        public void ProcessData(string data)
        {
            Console.WriteLine($"Processing data: {data}");
            OnDataProcessed?.Invoke($"Data '{data}' has been processed!");
        }
    }

    class EventSubscriber
    {
        public void HandleEvent(string message)
        {
            Console.WriteLine($"[Subscriber] Received: {message}");
        }
    }


    delegate int MathOperation(int x, int y);

    class Program
    {
        static int Multiply(int a, int b) => a * b;
        static int Add(int a, int b) => a + b;

        static void Main(string[] args)
        {
            Console.WriteLine("=== Delegates Demo ===");

            MathOperation op = Multiply;
            Console.WriteLine($"Multiply(4,5) = {op(4, 5)}");

            op = Add;
            Console.WriteLine($"Add(4,4) = {op(4, 4)}");

            Console.WriteLine("\n=== Event Demo ===");

            EventPublisher publisher = new EventPublisher();
            EventSubscriber subscriber = new EventSubscriber();

            publisher.OnDataProcessed += subscriber.HandleEvent;

            publisher.ProcessData("Student Record");
```

```
            publisher.ProcessData("Salary Data");

            Console.ReadLine();
        }
    }
}
```

F) Write the program with Exception handling

Project Name: ExceptionHandling

Project Type: Console App

Program Code:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace ExceptionHandling
{
    class Program
    {
        static void Main()
        {
            int a, b, result;

            try
            {
                Console.WriteLine("Enter Frist Number: ");
                a = Convert.ToInt32(Console.ReadLine());
                Console.WriteLine("Enter Second Number: ");
                b = Convert.ToInt32(Console.ReadLine());

                result = a / b;
                Console.WriteLine("Result=" + result);
            }

            catch(DivideByZeroException)
            {
                Console.WriteLine("Cannot divide by zero");
            }

            catch(FormatException)
            {
                Console.WriteLine("Invaild Input");
            }

            finally
            {
                Console.WriteLine("Program Ended");
            }
            Console.ReadLine();
        }
    }
}
```

Practical No 03

A) Create methods add(), multiply(), subtract(), divide, with suitable parameters and call these methods using concept of C# delegate.

Project Name: Arithmetic

Project Type: Console app

Program Code:

I.Filename: Program.cs

```
using System;

namespace Arithmatic
{
    public delegate void Calculation(int a, int b);
    class Arithmetic
    {
        public void Add(int a,int b)
        {
            Console.WriteLine("Addition=" + (a + b));
        }
        public void Subtract(int a,int b)
        {
            Console.WriteLine("Subtraction=" + (a - b));
        }
        public void Multiply(int a, int b)
        {
            Console.WriteLine("Multiplication=" + (a * b));
        }
        public void Divide(int a, int b)
        {
            Console.WriteLine("Division=" + (a / b));
        }
    }
    class Program
    {
        static void Main()
        {
            Arithmetic obj = new Arithmetic();
            Calculation c;
            c = obj.Add;
            c(20, 10);
            c = obj.Subtract;
            c(20, 10);
            c = obj.Multiply;
            c(20, 10);
            c = obj.Divide;
            c(20, 10);
            Console.ReadLine();
        }
    }
}
```

B) Write a program using multicast delegate

Project Name: Multicast

Project Type: Console App

Program Code:

I.Filename: Program.cs

```
using System;

public delegate void Demo();

class Test
{
    public void Show()
    {
        Console.WriteLine("show method called");
    }

    public void Display()
    {
        Console.WriteLine("display method called");
    }

    public void Print()
    {
        Console.WriteLine("print method called");
    }
}

class Program
{
    static void Main()
    {
        Test t = new Test();
        Demo d;

        d = t.Show;
        d += t.Display;
        d += t.Print;

        d();

        Console.ReadLine();
    }
}
```

C) Create a class BankAccount with AccountNumber and Balance. Implement property for Balance with validation (Balance cannot be negative).

Project Name: delc

Project Type: Console App

I.Filename: Program.cs

```
using System;
namespace delc
{
    class BankAccount
    {
        private double balance;

        public double Balance
        {
            get
            {
                return balance;
            }
            set
            {
                if (value >= 0)
                {
                    balance = value;
                }
                else
                {
                    Console.WriteLine("Balance cannot be negative");
                }
            }
        }
        public int AccountNumber { get; set; }
    }
    class Program
    {
        static void Main(string[] args)
        {
            BankAccount b = new BankAccount();
            b.AccountNumber = 101;
            b.Balance = 5000;
            Console.WriteLine("Account Number: " + b.AccountNumber);
            Console.WriteLine("Balance: " + b.Balance);
            b.Balance = -2000;

            Console.ReadLine();
        }
    }
}
```

Practical No 04

A) Design a Student Registration Form using the following ASP.NET controls:

i. Label and TextBox to accept Student Name
ii. RadioButton to select Gender (Male/Female)
iii. DropDownList to select Course (BSc IT, BSc CS, BCA)
iv. CheckBox for accepting Terms & Conditions
v. Calendar control to select Date
vi. Button control to submit the form
vii. Label control to display the output.

Project Name: StudentRegistration

Project Type: ASP.NET Web Application

Program Code:

I.Filename: StudentForm.aspx.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
using System.Web.UI;
using System.Web.UI.WebControls;

namespace StudentRegistration
{
    public partial class StudentForm : System.Web.UI.Page
    {
        protected void Page_Load(object sender, EventArgs e)
        {

        }

        protected void btnSubmit_Click(object sender, EventArgs e)
        {
            string gender = "";

            if(rbMale.Checked)
            {
                gender = "Male";
            }
            else if(rbFemale.Checked)
            {
                gender = "Female";
            }

            if (chkTerms.Checked)
            {
                lblOutput.Text = "StudentName :" + txtName.Text +
                    "<br/>Gender :" + gender+
                    "<br/>Course :" + ddlCourse.SelectedValue +
                    "<br/>Date :" + calDate.SelectedDate.ToShortDateString();
            }
            else
            {
                lblOutput.Text = "Please accept Terms & Condition";
            }
        }
    }
}
```

B) Create a Registration form to demonstrate use of various Validation controls.

Project Name: RegistrationForm

Project Type: ASP.NET Web Application

Program Code:

I. Filename:Registration.aspx.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
using System.Web.UI;
using System.Web.UI.WebControls;

namespace RegistrationForm
{
    public partial class Registration : System.Web.UI.Page
    {
        protected void Page_Load(object sender, EventArgs e)
        {
        }

        protected void btnSubmit_Click(object sender, EventArgs e)
        {
            if (Page.IsValid)
            {
                lblOutput.Text = "Registration Sucessful";
            }
        }
    }
}
```

C) Create Web Form to demonstrate use of Adrotator Control.

Project Name: AdrotatorDemo

Project Type: ASP.NET Web Application

Program Code:

I.Filename: Ads.xml

```
<?xml version="1.0" encoding="utf-8"?>
<Advertisements>
    <Ad>
        <ImageUrl>Images/p2.jpg</ImageUrl>
        <NavigateUrl>https://www.dell.com</NavigateUrl>
        <AlternateText>Dell Laptop</AlternateText>
        <Impressions>50</Impressions>
        <Keyword>Laptop</Keyword>
    </Ad>
</Advertisements>
```

II.Filename:AdrotatorDemo.aspx

```
<%@ Page Language="C#" AutoEventWireup="true"
   CodeBehind="AdrotatorDemo.aspx.cs"
   Inherits="Adrotator.AdrotatorDemo" %>

<!DOCTYPE html>

<html xmlns="http://w3.org">
<head runat="server">
    <title>AdRotator Control Demo</title>
</head>

<body>
    <form id="form1" runat="server">
        <div>
            <br />

            <asp:AdRotator
                ID="AdRotator1"
                runat="server"
                Height="300px"
                Width="300px"
                AdvertisementFile="~/Ads.xml" />

            <br />
            <br />
            <br />
        </div>
    </form>
</body>
</html>
```

Practical No 05

A) Create a web application to demonstrate the use of different types of Cookies.

Project Name: Cookies

Project Type: ASP.NET Web Application (.NET Framework)

Template: Web Forms

Program Code:

I.Filename: CookiesDemo.aspx.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
using System.Web.UI;
using System.Web.UI.WebControls;

namespace Cookies
{
    public partial class CookiesDemo : System.Web.UI.Page
    {
        protected void Page_Load(object sender, EventArgs e)
        {
        }

        protected void btnSave_Click(object sender, EventArgs e)
        {
            HttpCookie sessionCookie = new HttpCookie("SessionUser");
            sessionCookie.Value = txtName.Text;
            Response.Cookies.Add(sessionCookie);

            HttpCookie persistentCookie = new HttpCookie("PersistentUser");
            persistentCookie.Value = txtName.Text;
            persistentCookie.Expires = DateTime.Now.AddDays(7);
            Response.Cookies.Add(persistentCookie);

            lblOutput.Text = "cookies Saved Successfully";
        }

        protected void btnShow_Click(object sender, EventArgs e)
        {
            string sessionData = Request.Cookies["SessionUser"].Value;
            string persistentData = Request.Cookies["PersistentUser"].Value;

            lblOutput.Text = "Session Cookie: " + sessionData +
                             "<br/>Persistent Cookie: " + persistentData;
        }
    }
}
```

II.Filename: CookiesDemo.aspx

```
<%@ Page Language="C#" AutoEventWireup="true"
    CodeBehind="CookiesDemo.aspx.cs"
    Inherits="Cookies.CookiesDemo" %>
```

```
<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml">
<head runat="server">
    <title></title>
</head>
<body>
    <form id="form1" runat="server">
        <div>
            <asp:Label
                ID="Label1"
                runat="server"
                Text="User Name:">
            </asp:Label>
             
            <asp:TextBox
                ID="txtName"
                runat="server">
            </asp:TextBox>
            <br />
            <br />
            <asp:Button
                ID="btnSave"
                runat="server"
                OnClick="btnSave_Click"
                Text="Save cookie" />
            <br />
            <br />
            <asp:Button
                ID="btnShow"

                runat="server"
                OnClick="btnShow_Click"
                Text="Show Cookie" />
            <br />
            <br />
            <asp:Label
                ID="lblOutput"
                runat="server"
                Text="Output">
            </asp:Label>
            <br />
        </div>
    </form>
</body>
</html>
```

B) Create a Product Table in SQL Server and perform Insert Operation using Entity Framework Core.

Project Name: ProductEFCore

Project Type: ASP.NET Core Web App (MVC**)**

Program Code:

I.Filename: Products.cs

```
namespace ProductEFCore.Models
{
    public class Products
    {
        public int Id { get; set; }
        public required string ProductName { get; set; }
        public double Price { get; set; }
    }
}
```

II.Filename: AppDbContext.cs

```
using Microsoft.EntityFrameworkCore;
using ProductEFCore.Models;

namespace ProductEFCore.Data
{
    public class AppDbContext : DbContext
    {
        public AppDbContext(DbContextOptions<AppDbContext> options)

            : base(options)
        {
        }
        public DbSet<Products> Products { get; set; }
    }
}
```

II.Filename: Program.cs

```
using ProductEFCore.Data;
using Microsoft.EntityFrameworkCore;

var builder = WebApplication.CreateBuilder(args);

// Add services to the container.
builder.Services.AddControllersWithViews();

builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlServer(
        builder.Configuration.GetConnectionString("MyConnection")));

var app = builder.Build();

// Configure the HTTP request pipeline.
if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error");
    app.UseHsts();
}

app.UseHttpsRedirection();
app.UseRouting();

app.UseAuthorization();

app.MapStaticAssets();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}")
    .WithStaticAssets();

app.Run();
```

III.Filename: appsettings.json

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },

  "AllowedHosts": "*",
```

```
  "ConnectionStrings": {
    "MyConnection": "Server=(Localdb)\\MSSQLLocalDB; Database=ProductDB; Trusted_Connection=True;
TrustServerCertificate=True"
  }
}
```

IV.Filename: ProductsController.cs

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using ProductEFCore.Models;
using ProductEFCore.Data;

public class ProductsController : Controller
{
    private readonly AppDbContext _context;

    public ProductsController(AppDbContext context)
    {
        _context = context;
    }

    public async Task<IActionResult> Index()
    {
        return View(await _context.Products.ToListAsync());
    }

    public async Task<IActionResult> Details(int? id)
    {
        if (id == null)
        {
            return NotFound();
        }

        var products = await _context.Products
            .FirstOrDefaultAsync(m => m.Id == id);

        if (products == null)
        {
            return NotFound();
        }

        return View(products);
    }

    public IActionResult Create()
    {
```

```
        return View();
    }

    [HttpPost]
    [ValidateAntiForgeryToken]
    public async Task<IActionResult> Create(
        [Bind("Id,ProductName,Price")] Products products)
    {
        if (ModelState.IsValid)
        {
            _context.Add(products);
            await _context.SaveChangesAsync();
            return RedirectToAction(nameof(Index));
        }

        return View(products);
    }

    public async Task<IActionResult> Edit(int? id)
    {
        if (id == null)
        {
            return NotFound();
        }

        var products = await _context.Products.FindAsync(id);

        if (products == null)
        {
            return NotFound();
        }

        return View(products);
    }

    [HttpPost]
    [ValidateAntiForgeryToken]
    public async Task<IActionResult> Edit(
        int? id,
        [Bind("Id,ProductName,Price")] Products products)
    {
        if (id != products.Id)
        {
            return NotFound();
        }

        if (ModelState.IsValid)
        {
```

```
            try
            {
                _context.Update(products);
                await _context.SaveChangesAsync();
            }
            catch (DbUpdateConcurrencyException)
            {
                if (!ProductsExists(products.Id))
                {
                    return NotFound();
                }
                else
                {
                    throw;
                }
            }

            return RedirectToAction(nameof(Index));
        }

        return View(products);
    }

    public async Task<IActionResult> Delete(int? id)
    {
        if (id == null)
        {
            return NotFound();
        }

        var products = await _context.Products
            .FirstOrDefaultAsync(m => m.Id == id);

        if (products == null)
        {
            return NotFound();
        }

        return View(products);
    }

    [HttpPost, ActionName("Delete")]
    [ValidateAntiForgeryToken]
    public async Task<IActionResult> DeleteConfirmed(int? id)
    {
        var products = await _context.Products.FindAsync(id);

        if (products != null)
```

```
        {
            _context.Products.Remove(products);
            await _context.SaveChangesAsync();
        }

        return RedirectToAction(nameof(Index));
    }

    private bool ProductsExists(int? id)
    {
        return _context.Products.Any(e => e.Id == id);
    }
}
```

C) Develop a web application to perform CRUD Operations (Create, Read, Update, Delete) on Employee table using EF Core.

Project Name: AuthDemo

Project Type: ASP.NET Core Web App (MVC**)**

Program Code:

I.Filename: Employees.cs

```
namespace EmployeeCRUD.Models
{
    public class Employees
    {
        public int Id { get; set; }
        public string Name { get; set; }
        public string Department { get; set; }
        public double Salary { get; set; }
    }
}
```

II.Filename: AppDbContext.cs

```
using Microsoft.EntityFrameworkCore;
using EmployeeCURD.Models;

namespace EmployeeCURD.Data
{
    public class AppDbContext : DbContext
    {
        public AppDbContext(DbContextOptions<AppDbContext> options)
            : base(options)
        {
        }
        public DbSet<Employees> Employees { get; set; }
    }
}
```

III.Filename: Program.cs

```
using Microsoft.EntityFrameworkCore;
using EmployeeCURD.Data;

var builder = WebApplication.CreateBuilder(args);



builder.Services.AddControllersWithViews();

builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlServer(
        builder.Configuration.GetConnectionString("MyConnection")));

var app = builder.Build();

if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error");
    app.UseHsts();
}

app.UseHttpsRedirection();
app.UseRouting();

app.UseAuthorization();

app.MapStaticAssets();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}")
    .WithStaticAssets();

app.Run();
```

IV.Filename: appsettings.json

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*",
  "ConnectionStrings": {
```

```
    "MyConnection": "Server=(Localdb)\\MSSQLLocalDB; Database=EmployeeDB; Trusted_Connection=True;
TrustServerCertificate=True"
  }
}
```

V.Filename: EmployeesController.cs

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using EmployeeCRUD.Models;
using EmployeeCURD.Data;

public class EmployeesController : Controller
{
    private readonly AppDbContext _context;

    public EmployeesController(AppDbContext context)
    {
        _context = context;
    }



    public async Task<IActionResult> Index()
    {
        return View(await _context.Employees.ToListAsync());
    }

    public async Task<IActionResult> Details(int? id)
    {
        if (id == null)
        {
            return NotFound();
        }

        var employees = await _context.Employees
            .FirstOrDefaultAsync(m => m.Id == id);

        if (employees == null)
        {
            return NotFound();
        }

        return View(employees);
    }



    public IActionResult Create()
```

```
    {
        return View();
    }


    [HttpPost]
    [ValidateAntiForgeryToken]
    public async Task<IActionResult> Create(
        [Bind("Id,Name,Department,Salary")] Employees employees)
    {
        if (ModelState.IsValid)
        {
            _context.Add(employees);
            await _context.SaveChangesAsync();

            return RedirectToAction(nameof(Index));
        }

        return View(employees);
    }

    public async Task<IActionResult> Edit(int? id)
    {
        if (id == null)
        {
            return NotFound();
        }

        var employees = await _context.Employees.FindAsync(id);

        if (employees == null)
        {
            return NotFound();
        }

        return View(employees);
    }

    [HttpPost]
    [ValidateAntiForgeryToken]
    public async Task<IActionResult> Edit(
        int? id,
        [Bind("Id,Name,Department,Salary")] Employees employees)
    {
        if (id != employees.Id)
        {
            return NotFound();
        }
```

```
        if (ModelState.IsValid)
        {
            try
            {
                _context.Update(employees);
                await _context.SaveChangesAsync();
            }
            catch (DbUpdateConcurrencyException)
            {
                if (!EmployeesExists(employees.Id))
                {
                    return NotFound();
                }
                else
                {
                    throw;
                }
            }
            return RedirectToAction(nameof(Index));
        }
        return View(employees);
    }


    public async Task<IActionResult> Delete(int? id)
    {
        if (id == null)
        {
            return NotFound();
        }
        var employees = await _context.Employees
            .FirstOrDefaultAsync(m => m.Id == id);
        if (employees == null)
        {
            return NotFound();
        }
        return View(employees);
    }

    [HttpPost, ActionName("Delete")]
    [ValidateAntiForgeryToken]
    public async Task<IActionResult> DeleteConfirmed(int? id)
    {
        var employees = await _context.Employees.FindAsync(id);
        if (employees != null)
        {
            _context.Employees.Remove(employees);
        }
```

```
        await _context.SaveChangesAsync();
        return RedirectToAction(nameof(Index));
    }
    private bool EmployeesExists(int? id)
    {
        return _context.Employees.Any(e => e.Id == id);
    }
}
```

D) Authentication and Authorization in ASP.NET Core

Project Name: AuthDemo

Project Type: ASP.NET Core Web App (MVC**)**

Program Code:

I.Filename: Secret.cshtml

```
@{
    ViewData["Title"] = "Secret";
}
<h2>Authorized Users Only</h2>
```

II.Filename: HomeController.cs

```
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;

namespace AuthDemo.Controllers
{
    public class HomeController : Controller
    {
        public IActionResult Index()
        {
            return View();
        }

        [Authorize]
        public IActionResult Secret()
        {
            return View();
        }
    }
}
```

III.Filename: Program.cs

```
using AuthDemo.Data;
using Microsoft.AspNetCore.Identity;
```

```
using Microsoft.EntityFrameworkCore;



var builder = WebApplication.CreateBuilder(args);
// Add services to the container.
var connectionString = builder.Configuration.GetConnectionString("DefaultConnection")
    ?? throw new InvalidOperationException(
        "Connection string 'DefaultConnection' not found.");
builder.Services.AddDbContext<ApplicationDbContext>(options =>
    options.UseSqlServer(connectionString));
builder.Services.AddDatabaseDeveloperPageExceptionFilter();
builder.Services.AddDefaultIdentity<IdentityUser>(options =>
    options.SignIn.RequireConfirmedAccount = true)
    .AddEntityFrameworkStores<ApplicationDbContext>();
builder.Services.AddControllersWithViews();
var app = builder.Build();

// Configure the HTTP request pipeline.
if (app.Environment.IsDevelopment())
{
    app.UseMigrationsEndPoint();
}
else
{
    app.UseExceptionHandler("/Home/Error");
    app.UseHsts();
}
app.UseHttpsRedirection();
app.UseStaticFiles();
app.UseRouting();
app.UseAuthorization();
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");
app.MapRazorPages();
app.Run();
```